

คู่มือระบบ Thaisausage ERP Integration

Document version: 1.1.0

Date: 14 September 2026

Audience: ทีม IT / Operations, ทีม ERP และนักพัฒนาที่ดูแลระบบ

Code baseline: working tree ณ วันที่ 14 กันยายน 2026; การเปลี่ยนแปลงที่ยังไม่ commit ต้องถูก review และนำ
เข้าระบบ release ก่อนใช้งานจริง Production domain: <https://thaisausage.krs.co.th>

คู่มือนี้รวมข้อมูลจากการตรวจสอบโค้ดจริงทั้งหมดไว้ในเล่มเดียว ตั้งแต่การติดตั้ง การตั้งค่า API การ Deploy การ
ดูแลประจำวัน ไปจนถึงผลการตรวจสอบ โปรเจคและข้อจำกัดที่ต้องแก้ก่อนใช้งานจริง หากข้อความในเอกสารอื่นขัดกับ
คู่มือนี้ ให้ยึดตามโค้ดและคู่มือนี้ (ดูหัวข้อ 14.3)

1. ระบบนี้คืออะไร

Thaisausage เป็น middleware ที่คั่นกลางระหว่าง ERP กับ eVRP ERP แจ้งว่ามี Sales Order (SO) พร้อมส่ง
ระบบจะอ่าน SO ฉบับเต็ม ตรวจสอบถูกต้อง ก่อนการส่งซ้ำด้วย SQLite แล้วส่งต่อไปยัง eVRP (POST
/v1/orders/import) นอกจากนี้ระบบยังมีจุดรับ DO เข้าพื้นที่ staging เพื่อตรวจสอบ โดยจะไม่เขียนข้อมูลกลับ
ERP

```
ERP SQL Server <-- SELECT-only approved_orders_query (scheduled) --> map --> validate --+
ERP JSON -----> /erp/orders -----> validate --+
ERP REST <-GET-- /erp/pull -----> map --> validate -----+
                                                    v
                                submission guard (SQLite submissions + order_claims) --
> eVRP

ERP DO -----> /erp/do-received --> do_receipts (staged, never written back to ERP)
```

หลักการสำคัญ:

- ERP เป็นแหล่งข้อมูลหลักของ SO; ระบบใช้ Schedule และ query ที่กรองเฉพาะ `IsApprSo = 1`
- ระบบอ่าน SQL Server แบบ SELECT-only และไม่รับ SQL จาก HTTP request
- เมื่อ `dry_run=true` ระบบจะตรวจและคืน payload แต่ไม่ส่ง eVRP และไม่บันทึกการจอง request/order
- ระบบไม่ resend อัตโนมัติ ถ้าผลลัพธ์ไม่แน่นอน รายการจะเป็น `needs_review` และต้องให้คนตรวจ

2. สถานะปัจจุบัน (14 ก.ย. 2026)

ความสามารถ	มีในโค้ด	พร้อมใช้งานจริง
รับ webhook SO (/erp/hooks/order-ready)	มี	ใช่ — บันทึก trigger ได้
ส่ง SO แบบ JSON ตรง (/erp/orders)	มี	dry-run เท่านั้น จนกว่าจะผ่าน eVRP UAT
ดึง SO จาก ERP REST (/erp/pull)	มี	ต้องตั้ง <code>erp.base_url</code> และ <code>ERP_TOKEN</code> ก่อน
SQL Server connector	มีโค้ด	ยังรอ query/mapping ที่อนุมัติจาก ERP
Sweep worker แบบ scheduled approved-SO	มี	ปิดอยู่ (<code>sync.enabled=false</code>)
DO staging (/erp/do-received)	มี	ใช่ — staging เท่านั้น
DO writer เข้า <code>tbl_D0hdr</code> / <code>tbl_Dodtl</code>	มีโครงแล้ว (16 ก.ย.)	ไม่ — ปิดอยู่ (<code>do_write.enabled=false</code>) และยังไม่มีการ mapping จาก DBA
ส่งข้อมูลจริงไป eVRP	มี connector	ยังไม่ผ่าน UAT
Swagger UI / OpenAPI	มี	ใช่
COD callback, เขียนกลับ ERP, retry worker, outbox, API ปลด/แก้ SO	ไม่มี	—

ความคืบหน้าตามแผน `process/features/erp-sqlserver/` : Phase 01 (Discovery) อยู่ในสถานะ TESTING เพราะยังขาด ODBC driver และ SO ตัวอย่างที่ได้รับอนุมัติ ส่วน Phase 02–06 ยังไม่เริ่ม

Production ที่ติดตั้งอยู่ตาม Operations Guide คือ VPS `krs-cloud` ใช้ Docker Compose + Caddy และตั้ง `dry_run=true` , `sync.enabled=false`

3. โครงสร้างโปรเจค

Path	หน้าที่
thaisausage/__main__.py	จุดเริ่มโปรแกรม: โหลด config, สร้าง service, HTTP server และ thread <code>sql-sweep</code>
thaisausage/api.py	HTTP handler: auth, จำกัด body, routing, แปลง error เป็น HTTP status
thaisausage/service.py	business logic: submission guard, webhook inbox, DO staging, sweep และ SQLite schema
thaisausage/contracts.py	กติกา validate ของ standard SO JSON
thaisausage/connectors.py	HTTPS client ไป ERP REST และ eVRP รวมถึงฟังก์ชัน field mapping (mapped , lookup)
thaisausage/sqlserver.py	ODBC connection string, SELECT guard, <code>fetch_orders</code> และจัดกลุ่มแถวเป็น SO
thaisausage/do_writer.py	DO writer เข้า ERP: สร้าง INSERT แบบ parameterized, transaction เดียว, ปิดเป็นค่าเริ่มต้น
thaisausage/config.py	อ่าน <code>.env</code> และ config JSON พร้อมตรวจค่าบังคับ
config/example.json	แม่แบบ config (สำเนาจริงคือ <code>config/local.json</code> ซึ่งถูก ignore)
examples/erp-order.json	ตัวอย่าง request สำหรับ <code>/erp/orders</code>
docs/openapi.json	OpenAPI 3.0 ที่ใช้แสดงผลใน <code>/docs</code>
deploy/	Dockerfile, docker-compose.yml, Caddyfile และ <code>vps.env.example</code>
tests/	unittest: integration (HTTP + mock upstream) และ SQL config
process/	แผนงาน, รายการงาน phase และ context ของ workflow

ระบบใช้ Python 3.9 ขึ้นไปและ standard library เป็นหลัก ต้องใช้ `pyodbc` ร่วมกับ Microsoft ODBC Driver 18 เฉพาะเมื่อเชื่อม SQL Server (Docker image ติดตั้งให้แล้ว)

4. เริ่มใช้งานบนเครื่อง local

รันจาก root ของโปรเจค เพราะไฟล์ `.env` ถูกอ่านจาก working directory ปัจจุบัน:

```
cp config/example.json config/local.json
# สร้างไฟล์ .env อย่างน้อยต้องมี THAISAUSAGE_API_KEY
python3 -m thaisausage --config config/local.json
```

เมื่อเริ่มสำเร็จจะเห็นข้อความ `Thaisausage API http://0.0.0.0:8080 dry_run=True`

```
curl http://127.0.0.1:8080/health
curl -X POST http://127.0.0.1:8080/api/v1/erp/orders \
  -H "Authorization: Bearer $THAISAUSAGE_API_KEY" \
  -H 'Content-Type: application/json' \
  --data-binary @examples/erp-order.json
```

ระบบอ่าน config เฉพาะตอนเริ่ม ถ้าแก้ `config/local.json` หรือ `.env` ต้อง restart ทุกครั้ง

5. ตัวแปร environment (secrets)

Variable	ต้องใช้เมื่อ	หน้าที่
THAISAUUSAGE_API_KEY	ทุกกรณี (ถ้าไม่มี ระบบจะไม่เริ่ม)	Bearer key ที่ ERP ใช้เรียก API
VRP_TOKEN	<code>dry_run=false</code> (ถ้าไม่มี ระบบจะไม่เริ่ม)	ส่งไป eVRP ใน header <code>X-Token</code>
ERP_TOKEN	ใช้ <code>/erp/pull</code>	token สำหรับเรียก ERP REST
ERP_SQLSERVER_CONNECTION_STRING	sweep ทำงาน	ส่วน server/database/encryption ของ ODBC string
ERP_SQL_USER / ERP_SQL_PASSWORD	<code>sqlserver.auth_mode=sql</code>	credential แบบอ่านอย่างเดียว ระบบต่อท้ายให้ตอน connect

กติกาของ `.env` :

- ระบบรองรับรูปแบบ `KEY=value` บรรทัดที่ขึ้นต้นด้วย `#` เป็น comment และตัดเครื่องหมายคำพูดครอบคำออกให้
- ถ้า environment ของ shell หรือ container มีค่าอยู่แล้ว ค่านั้นมีสิทธิ์เหนือค่าในไฟล์
- `.env` และ `deploy/vps.env` อยู่ใน `.gitignore` ห้าม commit ห้ามใส่ในเอกสาร และห้ามถ่ายภาพหน้าจอที่มีค่าเหล่านี้
- ห้ามใส่ `UID` หรือ `PWD` ใน connection string เพราะระบบจะปฏิเสธ ให้ใช้ตัวแปรแยกแทน
- บน VPS ให้เก็บที่ `/opt/thaisausage/.env` และตั้ง permission เป็น `600`
- `deploy/vps.env` ใช้เก็บข้อมูล SSH ของผู้ดูแล (`VPS_HOST` , `VPS_PORT` , `VPS_USER` , `VPS_SSH_KEY`) และไม่ได้ถูกโหลดเข้า container

6. การตั้งค่า `config/local.json`

Key	ค่าเริ่มต้นใน example	ความหมาย
host , port	0.0.0.0 , 8080	address ที่ HTTP server รอรับ
database	data/integration.sqlite3	ไฟล์ SQLite (path สัมพันธ์กับ working directory)
api_key_env	THAISAUUSAGE_API_KEY	ชื่อตัวแปรที่เก็บ API key
dry_run	true	ต้องเป็น boolean เท่านั้น false คือส่งจริง
sync.enabled	false	เปิดหรือปิด SQL sweep
sync.interval_seconds	60	รอบ sweep สำหรับในกรณีที่ไม่มี webhook มาปลุก
vrp.base_url	https://vrp.oneplatformth.com/api	ต้องเป็น HTTPS
vrp.token_env , vrp.timeout_seconds	VRP_TOKEN , 30	token และ timeout ของ eVRP
erp.base_url , orders_path	ว่าง, /api/orders	ERP REST สำหรับ /erp/pull
erp.token_env , auth_header , auth_prefix	ERP_TOKEN , Authorization , Bearer	วิธีส่ง token ไป ERP
erp.orders_list_path , items_path	data.orders , items	ตำแหน่ง array ใน JSON ของ ERP
erp.field_map , item_field_map	map ชื่อเดียวกัน	key = field ปลายทาง, value = path ใน ERP
sqlserver.connection_string_env , username_env , password_env	ชื่อตามหัวข้อ 5	ชื่อตัวแปร secret
sqlserver.auth_mode	sql	sql หรือ integrated (Trusted_Connection)
sqlserver.connect_timeout_seconds , query_timeout_seconds	10 , 30	timeout ของ ODBC
sqlserver.approved_orders_query	ว่าง	SELECT ที่ผ่านการ review แล้ว ต้องมีเงื่อนไข IsApprSo = 1

Key	ค่าเริ่มต้นใน example	ความหมาย
sqlserver.order_key	order_no	ชื่อคอลัมน์ที่ใช้จัดกลุ่มแถวเป็น SO
sqlserver.field_map , item_field_map	ว่าง	map คอลัมน์ SQL ไปเป็น standard JSON
do_write.enabled	false	เปิดการเขียน DO เข้า ERP ห้ามเปิดจนกว่าจะผ่าน UAT และ sign-off
do_write.header_table , detail_table	ว่าง	ชื่อตารางปลายทางที่ผ่านการ review
do_write.connection_string_env , username_env , password_env	ERP_SQLSERVER_WRITE_* , ERP_SQL_WRITE_*	credential สำหรับเขียน ต้องคนละชุดกับบัญชี read-only
do_write.header_columns , detail_columns	ว่าง	mapping รายคอลัมน์ (ดู docs/do-insert-contract.md)
do_write.detail_line_start	1	เลขเริ่มต้นของ Slno

Key ต่อไปนี้อยู่ใน example แต่โค้ดยังไม่ได้อ่าน จึงยังไม่มีผลกับการทำงานจริง: `source.*` , `sqlserver.driver` , `sqlserver.encrypt` , `sqlserver.trust_server_certificate` , `sqlserver.approved_objects` , `sqlserver.approved_query_set` ค่า driver และ encryption ที่ใช้จริงจะมาจาก `ERP_SQLSERVER_CONNECTION_STRING` เท่านั้น

Mapping รองรับ path ซ้อนกันด้วยจุด เช่น `"customer.code": "buyer.customer_code"` ถ้าไม่พบ path ระบบจะใส่ค่า `null` ระบบจะไม่แปลงชนิดข้อมูล ภาษี ส่วนลด หรือหน่วยให้

7. API reference

7.1 การยืนยันตัวตนและข้อจำกัดทั่วไป

Endpoint สาธารณะที่ไม่ต้องใช้ key ได้แก่ `GET/HEAD /health` , `GET/HEAD /openapi.json` , `GET/HEAD /docs` และ `GET /favicon.ico` (ตอบ 204) ส่วน endpoint อื่นทั้งหมดต้องส่ง header ดังนี้:

```
Authorization: Bearer <THAISAUSAGE_API_KEY>
Content-Type: application/json
```

ข้อจำกัดของ POST ทุกตัว:

- `Content-Type` ต้องเป็น `application/json` และต้องมี `Content-Length` ขนาด 1 ถึง 1,048,576 bytes
- ไม่รองรับ `Transfer-Encoding` (chunked) หากส่งมาจะได้ 422
- body ต้องเป็น JSON object และห้ามมี `NaN` หรือ `Infinity`

- connection มี timeout 35 วินาที
- ระบบไม่เขียน access log เพื่อไม่ให้ข้อมูลลูกค้าหรือ header รั่วไหล

7.2 สรุป endpoint

Method / Path	Auth	สำเร็จ	หน้าที่
GET /health	ไม่ ต้อง	200	{status, dry_run} ตรวจสอบว่า process ทำงานอยู่ ไม่ได้ตรวจ SQL หรือ eVRP
GET /docs	ไม่ ต้อง	200	Swagger UI (โหลดไฟล์จาก unpkg.com)
GET /openapi.json	ไม่ ต้อง	200	OpenAPI contract
POST /api/v1/erp/hooks/order-ready	ต้อง	202	บันทึก event และปลุก sweep worker
POST /api/v1/erp/orders	ต้อง	200/202	รับ standard SO JSON แล้ว validate และส่งต่อ
POST /api/v1/erp/pull	ต้อง	200/202	ดึง ERP REST หนึ่งครั้ง แล้ว map, validate และส่งต่อ
POST /api/v1/erp/do-received	ต้อง	202	เก็บ DO เข้า staging
GET /api/v1/submissions/{request_id}	ต้อง	200/404	ผลที่บันทึกไว้ของการส่งจริง

7.3 Webhook: `POST /api/v1/erp/hooks/order-ready`

```
{
  "event_id": "ERP-SO-20260912-000001",
  "event_type": "sales_order.ready",
  "source_id": "main-erp",
  "company_id": "THAI",
  "order_no": "SO-0001",
  "changed_at": "2026-09-12T10:00:00+07:00"
}
```

Field	บังคับ	กติกา
event_id	ใช่	ไม่ว่างและยาวไม่เกิน 120 ตัว ควรใช้เฉพาะ A-Z a-z 0-9 . _ : - (ดู F5)
event_type	ใช่	sales_order.ready หรือ sales_order.changed
source_id	ใช่	รหัสแหล่ง ERP
order_no	ไม่บังคับใน API แต่ควรส่ง ทุกครั้ง	ใช้เป็นพารามิเตอร์ของ SQL query (ดู F4)
company_id, changed_at	ไม่	บันทึกไว้เพื่อใช้ตรวจสอบ ปัจจุบัน sweep ยังไม่ได้ใช้

ครั้งแรกระบบจะตอบ 202 {"event_id", "state": "received", "sweep": "queued"} ถ้าส่ง event_id เดิมพร้อม body เดิมซ้ำจะได้ 202 {"event_id", "state": "สถานะปัจจุบัน", "replayed":

true} หาก body ต่างกันจะตอบ 409 และไม่ดำเนินการซ้ำ ทุกการแก้ไข SO จึงต้องใช้ event_id ใหม่
การตอบ 202 หมายความว่าบันทึก trigger แล้วเท่านั้น ไม่ได้แปลว่า SO ถึง eVRP แล้ว

7.4 ส่ง SO ตรง: `POST /api/v1/erp/orders`

ดูตัวอย่างเต็มได้ใน `examples/erp-order.json` กติกา validate มีดังนี้
(`thaisausage/contracts.py`):

Field	กติกา
<code>request_id</code>	1-120 ตัว ใช้ได้เฉพาะ A-Z a-z 0-9 . _ : -
<code>orders</code>	array จำนวน 1-100 รายการ
<code>order_no</code>	บังคับ ยาวไม่เกิน 100 และห้ามซ้ำใน batch
<code>order_date</code>	YYYY-MM-DD แบบ string
<code>delivery_date</code>	null, "" หรือ YYYY-MM-DD
<code>payment_in_day</code>	null หรือตัวเลขที่ ≥ 0 โดย 0 หมายถึง COD
<code>pickup_hub_code</code>	บังคับ
<code>delivery_point_code</code> / <code>shipping_address</code>	ต้องมีอย่างน้อยหนึ่งค่า
<code>customer.code</code>	บังคับ ยาวไม่เกิน 50
<code>items</code>	array จำนวน 1-2000 รายการ
<code>items[].item_code</code>	บังคับ ยาวไม่เกิน 50
<code>items[].quantity</code>	JSON number ที่ > 0
<code>items[].unit_price</code>	JSON number ที่ ≥ 0 (สินค้าแถมใช้ราคา 0 และแยกบรรทัด)
<code>items[].cbm</code> , <code>nw</code>	null หรือตัวเลขที่ ≥ 0

Field ที่ระบบยังไม่ validate ได้แก่ `customer.name`, `description`, `pickup_hub_name` และหน่วยสินค้า eVRP จะตรวจ master data อีกครั้งเมื่อส่งจริง ตัวเลขที่ส่งมาเป็น string เช่น "100" จะถูกปฏิเสธ

ผลลัพธ์:

- dry-run ตอบ 200 {state: "validated", dry_run: true, order_count, mapped_payload} โดยไม่บันทึกอะไรลงฐานข้อมูล
- live ตอบ 200 {state: "sent", vrp: {...}} เมื่อ eVRP ยืนยัน success: true หรือ 202 {state: "needs_review", reason} เมื่อผลไม่แน่นอน
- ส่ง request_id เดิมพร้อม payload เดิมจะได้ผลเดิมกลับมาพร้อม replayed: true
- request_id เดิมที่มี payload ต่างกัน หรือ order_no ที่เคยถูกจองภายใต้ request_id อื่น จะได้ 409 และระบบจะไม่ส่งทั้ง batch

7.5 ดึงจาก ERP REST: `POST /api/v1/erp/pull`

```
{"request_id": "ERP-PULL-0001", "query": {"page": 1}}
```

`query` ต้องเป็น object ที่มีแต่ค่า scalar ระบบจะแปลงเป็น query string แล้ว GET ไป `erp.orders_path` หนึ่งครั้ง ไม่มีการไล่น้ำอัดโนมิตี ถ้าไม่มีรายการจะตอบ `200 {state: "empty"}` ถ้ามีรายการระบบจะทำงานเหมือน `/erp/orders` ถ้าเรียก ERP ไม่สำเร็จจะได้ `502` ข้อควรระวังคือเมื่ออยู่ใน dry-run ระบบก็ยังเรียก ERP จริง

7.6 DO staging: `POST /api/v1/erp/do-received`

ต้องมี `receipt_id` หรือ `do_no` (ยาวไม่เกิน 120) ระบบเก็บ payload เติมพร้อม hash แล้วตอบ `202 {receipt_id, state: "staged", erp_write: false}` ถ้าส่ง receipt เติมพร้อม payload เติมจะได้ `replayed: true` ถ้า payload ต่างกันจะได้ `409` รูปแบบของ field ภายใน `data` ยังรอผลสำรวจจาก ERP endpoint นี้ไม่เขียน ERP ไม่ว่ากรณีใด การเขียน DO เข้า ERP เป็นงานแยกที่ต้องเรียกภายในเท่านั้น (ดูหัวข้อ 9.1) และยังไม่มี endpoint สาธารณะ

7.7 ตรวจสอบสถานะ: `GET /api/v1/submissions/{request\_id}`

endpoint นี้คืนผลที่บันทึกไว้ใน SQLite ซึ่งมีเฉพาะการส่งในโหมด live ดังนั้นในโหมด dry-run จะได้ `404` เสมอ

7.8 HTTP status

HTTP	ความหมาย	สิ่งที่ผู้เรียกควรทำ
200	validated / sent / empty / replay ของผลที่เสร็จแล้ว	บันทึกผล
202	บันทึก hook หรือ DO แล้ว หรือผลเป็น sending / needs_review	ห้ามถือว่าส่งถึง eVRP แล้ว
401	key ไม่มีหรือไม่ถูกต้อง	แก้ credential แล้วส่ง identity เดิมซ้ำ
404	path ไม่ถูกต้อง หรือไม่พบ submission	ตรวจ URL หรือ request_id
409	identity เดิมแต่ข้อมูลต่าง	หยุด automation แล้ว reconcile
422	JSON, header หรือ field ไม่ถูกต้อง	แก้ payload ถ้าข้อมูลเปลี่ยนให้ใช้ identity ใหม่
502	เรียก ERP REST ไม่สำเร็จ (<code>/erp/pull</code>)	retry ตามนโยบายของ operator
500	internal error	แจ้งผู้ดูแลระบบ

7.9 สถานะ submission และ hook

State	ใช้กับ	ความหมาย
received	hook	บันทึกแล้ว รอ sweep
validated	submission, hook	ผ่าน validate ใน dry-run โดยยังไม่ได้ส่งจริง
sending	submission, hook	จองแล้ว กำลังส่ง ถ้าค้างอยู่แสดงว่า process หยุดกลางทาง ต้องตรวจเอง
sent	submission, hook	eVRP ตอบ success: true
needs_review	submission, hook	ผลไม่แน่นอน: upstream_response_not_confirmed , upstream_http_NNN , upstream_outcome_unknown , connector_error
review	hook	sweep ไม่พบ SO หรือเกิด error (ระบบไม่ได้เก็บเหตุผลไว้ ดู F6)
staged	DO	เก็บเข้า staging แล้ว

8. การทำงานของ sweep worker

thread `sql-sweep` เริ่มพร้อม HTTP server และทำงานเป็นรอบดังนี้:

- รอจนครบ `sync.interval_seconds` หรือจนกว่าจะมี webhook ใหม่มาปลุก ซึ่งจะตื่นทันที
- ถ้า `sync.enabled=false` ระบบข้ามรอบนั้นไป ปิดอยู่ก็ยังไม่รับ webhook ได้ตามปกติ
- ระบบดึง hook ที่มีสถานะ `received` เรียงตามเวลาที่รับ ครั้งละไม่เกิน 100 รายการ
- ทุก Schedule เรียก `approved_orders_query` โดยไม่รับ SQL จาก HTTP และส่งเฉพาะ SO ที่ query กรองว่า Approve แล้ว
- ถ้าพบ ระบบจะส่งเข้า submission guard ด้วย `request_id = "HOOK-" + event_id[:110]` แล้วอัปเดตสถานะ hook ตามผล
- error ใดๆ ใน hook นั้นจะทำให้ hook เป็น `review` ส่วน error ระดับทั้งรอบจะถูกกลืนเงียบๆ และไม่มี log

พฤติกรรมที่ผู้ดูแลต้องรู้:

- ถ้า `order_no` เดิมถูกส่งไปแล้ว event `sales_order.changed` จะชนการจอง `order_no` และกลายเป็น `review` ในกรณีนี้ต้องแก้ไข SO ที่ eVRP ด้วยมือ เพราะยังไม่มี contract สำหรับ update หรือ cancel
- ถ้ามี hook ค้างเกิน 100 รายการ ระบบจะเก็บกวาดรอบละ 100 ต่อหนึ่งการปลุกหรือหนึ่ง interval
- เปิด `sync.enabled=true` ใน `dry_run=true` ได้เฉพาะกับ SO ที่ได้รับอนุมัติ เพราะระบบจะ preview และคง Hook เป็น `received` พร้อม reason `dry_run_preview`; ก่อน live ต้องตรวจ mapping ให้เสร็จ (ดู F2)

9. SQL Server connector

การประกอบ connection string: ระบบนำ ERP_SQLSERVER_CONNECTION_STRING มาต่อท้ายด้วย UID={user};PWD={password} (escape ค่าด้วยวงเล็บปีกกา) เมื่อใช้ auth_mode=sql หรือต่อท้าย Trusted_Connection=yes เมื่อใช้ integrated

```
DRIVER={ODBC Driver 18 for SQL Server};SERVER=erp-  
db.example,1433;DATABASE=ERP;Encrypt=yes;TrustServerCertificate=no;Connection Timeout=10
```

SELECT guard ใน select_approved ยอมรับเฉพาะคำสั่งที่ขึ้นต้นด้วย SELECT และปฏิเสธคำว่า INSERT, UPDATE, DELETE, MERGE, EXEC, EXECUTE, ALTER, DROP, TRUNCATE และ INTO guard นี้ตรวจเพียงข้อความ ไม่ใช้กลไกความปลอดภัยหลัก บัญชี SQL จึงต้องมีสิทธิ์ SELECT เฉพาะ view ที่อนุมัติเท่านั้น (ดู F11)

การแปลงแถว SQL เป็น SO:

- แถวผลลัพธ์จะถูกจัดกลุ่มตามคอลัมน์ order_key โดยใช้แถวแรกของกลุ่ม map header ด้วย field_map
- ทุกแถวถูก map ด้วย item_field_map และรวมเป็นรายการสินค้าเมื่อ item_code ไม่เป็น null
- ชื่อคอลัมน์ต้องไม่มีจุด เพราะจุดใช้แบ่ง path ให้ตั้ง alias ใน SQL แทน
- connector จะแปลง date / datetime เป็น ISO string และ Decimal เป็นตัวเลข JSON ให้ตรง contract; ยังต้องยืนยันผลกับ SQL จริงแบบ read-only (ดู F3)

ตัวอย่างโครง config (ชื่อ view และคอลัมน์เป็นตัวอย่าง ต้องแทนด้วยผล Phase 01):

```
"sqlserver": {  
  "orders_query": "SELECT h.so_no AS order_no, CONVERT(char(10), h.so_date, 23) AS  
order_date, h.credit_days AS payment_in_day, h.cust_code AS customer_code, h.ship_addr AS  
shipping_address, h.hub_code AS pickup_hub_code, l.line_no AS line_no, l.item_code AS  
item_code, l.item_name AS description, CAST(l.qty AS float) AS quantity, CAST(l.price AS  
float) AS unit_price FROM dbo.v_so_ready h JOIN dbo.v_so_lines l ON l.so_no = h.so_no WHERE  
h.so_no = ?",  
  "order_key": "order_no",  
  "field_map": {"order_no": "order_no", "order_date": "order_date", "payment_in_day":  
"payment_in_day", "customer.code": "customer_code", "shipping_address": "shipping_address",  
"pickup_hub_code": "pickup_hub_code"},  
  "item_field_map": {"line_no": "line_no", "item_code": "item_code", "description":  
"description", "quantity": "quantity", "unit_price": "unit_price"}  
}
```

ตรวจการเชื่อมต่อแบบอ่าน metadata อย่างเดียว (ไม่แตะตารางธุรกิจ):

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import json; from  
thaisausage.sqlserver import SQLServerConnector;  
c=json.load(open('/app/config/local.json'));  
print(SQLServerConnector(c['sqlserver']).check_read_only_connection())"
```

9.1 DO writer เข้า ERP (`do\_write`)

สถานะ:  CODE DONE — โครงสร้างพร้อมและมี test แต่ยังไม่มี mapping จริงและยังไม่เคยเขียน ERP รายละเอียด mapping ทั้งหมดอยู่ใน docs/do-insert-contract.md

กติกความปลอดภัยที่บังคับในโค้ด:

- ค่าเริ่มต้น `do_write.enabled=false` เปิดได้จาก config เท่านั้น ห้ามเปิดจาก payload และยังไม่มี endpoint สาธารณะ
- ต้องใช้ credential คนละชุดกับบัญชี read-only ถ้าตั้ง env เดียวกับ `ERP_SQL_USER / ERP_SQL_PASSWORD / ERP_SQLSERVER_CONNECTION_STRING` ระบบจะปฏิเสธ
- validate mapping และ payload ให้เสร็จก่อนเปิด connection เสมอ
- ชื่อตารางและคอลัมน์มาจาก config ที่ review แล้ว ค่าทุกค่าถูกผูกเป็น parameter ไม่มีการต่อ string จาก payload
- Header และ Detail อยู่ใน transaction เดียว ถ้าบรรทัดใดล้มเหลวจะ rollback ทั้งชุด
- `EntryDate` ใช้ `GETDATE()` ของ SQL Server ส่วน `IsAcc=0` , `IsAccBy` , `IsAccDate` , `DocuNw` เป็นค่าคงที่ตาม contract
- เมื่อ `dry_run=true` ระบบจะ validate และนับ statement เท่านั้น ไม่เปิด connection

สถานะที่บันทึกในตาราง `do_writes` :

State	ความหมาย	เขียนซ้ำได้
preview	dry-run ตรวจ mapping แล้ว ไม่ได้เขียน	ได้
disabled	flag ปิดอยู่	ได้
rejected	validate ไม่ผ่าน ไม่มี SQL ถูกส่ง	ได้ (หลังแก้สาเหตุ)
inserted	commit สำเร็จ	ไม่ได้
needs_review	ผลไม่แน่นอน เช่น timeout ระหว่าง commit	ไม่ได้ ต้องตรวจ ERP ด้วยมือก่อน

การกันซ้ำใช้ `receipt_id` เป็น primary key พร้อม unique index บน `do_no` และ `transaction_no` ถ้า payload ของ receipt เดิมเปลี่ยนหลังเคยเขียนแล้วจะได้ conflict ไม่ใช้การเขียนซ้ำ

10. ข้อมูลใน SQLite

```
submissions(request_id PK, payload_hash, state, result JSON, created_at)
order_claims(order_no PK, request_id)
erp_hooks(event_id PK, event_type, source_id, company_id, order_no, changed_at, state, received_at)
do_receipts(receipt_id PK, do_no, payload_hash, payload, state, received_at)
do_writes(receipt_id PK, do_no UNIQUE, transaction_no UNIQUE, payload_hash, state, reason, updated_at)
```

ใน container ไฟล์ฐานข้อมูลอยู่ที่ `/app/data/integration.sqlite3` บน volume `thaisausage-data` ตาราง `submissions` เก็บเฉพาะ hash, สถานะ และเลขอ้างอิงจาก eVRP ไม่ได้เก็บ payload เต็ม ดังนั้นจึงต้องเก็บไว้ที่ ERP เพื่อใช้ reconcile

ดูจำนวนรายการตามสถานะ:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3; db=sqlite3.connect('data/integration.sqlite3'); print('hooks', db.execute('SELECT state, count(*) FROM erp_hooks GROUP BY state').fetchall()); print('submissions', db.execute('SELECT state, count(*) FROM submissions GROUP BY state').fetchall())"
```

สำรองข้อมูลโดยใช้ SQLite backup API ซึ่งทำได้ขณะระบบทำงาน:

```
docker compose -f deploy/docker-compose.yml exec thaisausage python -c "import sqlite3; sqlite3.connect('data/integration.sqlite3').backup(sqlite3.connect('/tmp/backup.sqlite3'))"
docker compose -f deploy/docker-compose.yml cp thaisausage:/tmp/backup.sqlite3 ./integration-$(date +%F).sqlite3
```

ห้ามลบแถวใน `order_claims` เพื่อบังคับส่งซ้ำ และห้าม restore backup เก้าทับขณะเปิดโหมด live ก่อน reconcile กับ eVRP

11. Deploy บน VPS

Service	รายละเอียด
thaisausage	Python 3.11 slim + ODBC Driver 18 รันด้วย user <code>appuser</code> (uid 10001) เปิดพอร์ต 8080 เฉพาะใน network ภายใน root filesystem เป็น read-only, <code>/tmp</code> เป็น tmpfs และมี healthcheck เรียก <code>/health</code> ทุก 30 วินาที
caddy	Caddy 2 รับพอร์ต 80/443 ออก HTTPS อัตโนมัติสำหรับ <code>thaisausage.krs.co.th</code> แล้ว reverse proxy ไปที่ app และเริ่มหลังจาก app healthy แล้ว
Volumes	<code>thaisausage-data</code> (SQLite), <code>caddy-data</code> , <code>caddy-config</code>
Mount	<code>config/local.json</code> แบบ read-only และ <code>.env</code> ผ่าน <code>env_file</code> (format raw)

ข้อกำหนดก่อน deploy:

- DNS A record ของ `thaisausage.krs.co.th` ต้องชี้มาที่ VPS และพอร์ต 80/443 ต้องเปิดเพื่อให้ออก certificate ได้
- image ถูก build สำหรับ amd64 เพราะ repo ของ Microsoft ระบุ `arch=amd64` ไว้ตายตัว
- ต้องมี `/opt/thaisausage/.env` (permission 600) และ `config/local.json` บน VPS

Deploy ครั้งแรกหรืออัปเดต:

```
cd /opt/thaisausage
git fetch origin main
git reset --hard origin/main      # ทั้งการแก้ไขไฟล์ที่ track บน VPS; .env และ config/local.json
ถูก ignore จึงไม่หาย
docker compose -f deploy/docker-compose.yml build
docker compose -f deploy/docker-compose.yml up -d
docker compose -f deploy/docker-compose.yml ps
curl -fsS https://thaisausage.krs.co.th/health
```

ก่อนอัปเดตให้สำรอง SQLite (หัวข้อ 10) เสมอ หลังอัปเดตให้ตรวจ `/health` , `/docs` , log และลองส่ง webhook กับ DO แบบ mock โค้ดที่ยังไม่ commit และ push จะไม่ขึ้นไปบน VPS ดังนั้น ณ วันที่เขียนเอกสาร ระบบบน production ยังไม่มีการปลุก sweep ด้วย webhook, การรองรับ HEAD, favicon และการแก้ CSP ของ Swagger

12. การดูแลประจำวันและการแก้ปัญหา

12.1 รายการตรวจประจำวัน

- [] `docker compose ps` ต้องแสดงทั้งสอง service เป็น running และ app เป็น healthy
- [] `https://thaisausage.krs.co.th/health` ต้องตอบ 200 และค่า `dry_run` ต้องตรงกับที่อนุมัติไว้
- [] นับ hook ที่เป็น `received` ค้างนาน และ hook ที่เป็น `review`
- [] นับ submission ที่เป็น `sending` หรือ `needs_review`
- [] ตรวจพื้นที่ดิสก์ของ volume และตรวจว่ามี backup ล่าสุด
- [] ตรวจอายุ certificate ของ Caddy

12.2 อาการและวิธีแก้

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
ระบบไม่เริ่ม: <code>dry_run must explicitly be true or false</code>	ค่าใน config ไม่ใช่ boolean	แก้เป็น <code>true</code> หรือ <code>false</code> โดยไม่มีเครื่องหมายคำพูด
ระบบไม่เริ่ม: Set the environment variable named by <code>api_key_env</code>	ไม่มี <code>THAISAUUSAGE_API_KEY</code>	ตรวจ <code>.env</code> หรือ <code>env_file</code> แล้ว restart
ระบบไม่เริ่ม: Live mode requires the VRP token	<code>dry_run=false</code> แต่ไม่มี <code>VRP_TOKEN</code>	ใส่ token หรือกลับไปใช้ dry-run
ระบบไม่เริ่ม: <code>KeyError</code>	config ขาด key หลัก เช่น <code>vrp</code> หรือ <code>erp</code>	เทียบกับ <code>config/example.json</code>
401 unauthorized	key ผิด หรือขาดคำว่า <code>Bearer</code>	ตรวจ header
422 Content-Type must be application/json	client ส่ง form หรือ chunked	ตั้ง header และส่ง Content-Length
502 <code>erp_request_failed</code>	ERP REST ไม่ตอบ, ไม่ใช่ HTTPS หรือเจอ redirect	ตรวจ <code>erp.base_url</code> และ token (ระบบไม่ follow redirect)
hook ค้างที่ <code>received</code>	<code>sync.enabled=false</code> หรือ sweep error ทั้งรอบ	เปิด sync ตาม gate และตรวจ SQL config และ driver
hook เป็น <code>review</code>	ไม่พบ SO, SQL error, validate ไม่ผ่าน, <code>order_no</code> ซ้ำ, <code>order_no</code> หาย หรือข้อมูลเก่าที่ไม่ทราบ hash	ทดสอบ query ด้วย <code>order_no</code> นั้นแบบ read-only แล้วตรวจ reason/mapping
submission เป็น <code>needs_review</code>	eVRP timeout, error หรือตอบไม่ตรงรูปแบบ	ตรวจที่ eVRP ด้วยเลข SO ก่อน ห้ามส่ง payload ที่เปลี่ยนแล้วซ้ำ
<code>/docs</code> หน้าว่าง	browser โหลด unpkg.com ไม่ได้	ใช้ <code>/openapi.json</code> กับ Postman หรือ Swagger Editor แทน
app unhealthy	process ค้าง หรือ config ผิด	<code>docker compose logs --tail=100 thaisausage</code>
HTTPS ใช้ไม่ได้	DNS ผิด หรือพอร์ต 80/443 ถูกปิด	ตรวจ DNS, firewall และ <code>docker compose logs caddy</code>

12.3 ขั้นตอนเปิดใช้งานจริง (ย่อ)

- Gate A: ติดตั้ง ODBC แล้ว, ผ่าน metadata check (หัวข้อ 9) และบัญชี SQL เป็น read-only จริง
- Gate B: ใส่ `approved_orders_query` และ mapping แล้ว ทดสอบกับ SO ที่อนุมัติใน dry-run ต้องได้ JSON ถูกต้องครบทุกกรณี (COD, credit, ของแถม, หลายบรรทัด, ภาษาไทย, ทศนิยม)
- Gate C: ทดสอบกับ eVRP UAT โดยใช้ token ทดสอบ แล้วตรวจเลขอ้างอิงและยอดเงิน
- Gate D: ซ่อม backup/restore, กำหนดผู้รับผิดชอบ แล้วตั้ง `dry_run=false` ก่อน จากนั้นจึงตั้ง `sync.enabled=true` และเฝ้าดูรอบแรก

- F1–F3 แก้แล้วและมี regression test; Gate B ยังต้องผ่านการเชื่อม SQL จริงแบบ read-only กับ SO ที่อนุมัติ รายละเอียดอยู่ใน Operations Guide และ Integration Test Plan

13. การทดสอบ

```
python3 -m unittest discover -s tests -v
```

ผลวันที่ 16 ก.ย. 2026: ผ่าน 63 จาก 63 test ครอบคลุม HTTP auth, validation, idempotency, การส่งซ้ำพร้อมกัน, timeout, DO staging, hook, credential ของ SQL และชุดทดสอบ DO writer (parameter binding, transaction, rollback, duplicate, dry-run และ flag ปิด) ทุก test ใช้ mock โดยไม่เรียก ERP หรือ eVRP จริง และไม่มี INSERT เข้า ERP เกิดขึ้น

ส่วนที่ยังไม่มี test เชื่อม SQL Server จริงคือ connection/query กับ schema ของ ERP; มี unit test สำหรับ `_group_rows` และชนิดข้อมูล `Decimal/date` แล้ว แต่ยังต้องทำ read-only verification กับ SO ที่อนุมัติ

14. ผลการตรวจสอบโปรเจค

14.1 ประเด็นที่ตรวจพบและสถานะแก้ไข

ID	ระดับ	ประเด็น	ตำแหน่ง	ผลกระทบ / หลักฐาน	ข้อเสนอ
F1	สูง	mapped ไม่ได้ import	thaisausage/sqlserver.py	แก้แล้ว: import โดยตรงและมี regression test <code>_group_rows</code>	
F2	สูง	sweep ใน dry-run ทำให้ hook ถูกใช้ไปแล้ว	service.py	แก้แล้ว: dry-run คง Hook เป็น <code>received</code> พร้อม reason <code>dry_run_preview</code>	
F3	สูง	ชนิดข้อมูลจาก pyodbc (Decimal, date) ไม่ผ่าน validate	sqlserver.py	แก้แล้ว: normalize เป็น JSON-compatible type และมี unit test; ยังต้องทดสอบ SQL จริงแบบ read-only	
F4	กลาง	hook ที่ไม่มี <code>order_no</code> ทำให้ query ไม่มีเงื่อนไข	service.py	แก้แล้ว: ตั้ง <code>review/order_no_required</code> และไม่เรียก SQL	
F5	กลาง	ชุดอักขระของ <code>event_id</code> ไม่ตรงกับ <code>request_id</code>	service.py	แก้แล้ว: validate ด้วย pattern เดียวกับ <code>request_id</code>	
F6	กลาง	sweep error ถูกกลืนเงิบๆ และไม่เก็บ reason	__main__.py, service.py	แก้แล้ว: log แบบไม่เปิดเผย payload และเก็บ reason ต่อ Hook	
F7	กลาง	hook ข้างไม่ได้เทียบ body	service.py	แก้แล้ว: เก็บ payload hash และตอบ 409 เมื่อ <code>event_id</code> เดิมมีข้อมูลต่างกัน	
F8	ต่ำ	config หลาย key ไม่มีผลกับการทำงาน	config/example.json	ทำให้เข้าใจผิดว่ามีการควบคุม เช่น <code>encrypt</code> หรือ <code>approved_objects</code>	implement การตรวจหรือทำเครื่องหมายว่ายังไม่ใช้
F9	ต่ำ	โค้ดและเอกสารยังไม่ commit	api.py, __main__.py, docs/	production ต่างจาก repo และเอกสารยังไม่อยู่ใน Git	review แล้ว commit และ deploy
F10	ต่ำ	Swagger โหลดจาก unpkg	api.py	แก้แล้ว: pin เป็น 5.32.15 ; ยังควรพิจารณา vendor asset/SRI ใน hardening รอบถัดไป	
F11	ต่ำ	SELECT guard เป็นการตรวจข้อความ	sqlserver.py (select_approved)	แก้แล้ว: ปฏิเสธ INTO เพิ่มเติม และยังต้องพึงสิทธิ์ read-only ของ DB	

14.2 จุดแข็งที่พบ

- ใช้ idempotency แบบ transaction (`BEGIN IMMEDIATE`) มี test การส่งซ้ำพร้อมกัน และไม่ resend อัตโนมัติเมื่อผลไม่แน่นอน
- เทียบ API key แบบ constant-time, จำกัดขนาด body, ปฏิเสธ `NaN` และปิด access log เพื่อป้องกันข้อมูลรั่ว
- connector บังคับ HTTPS, ไม่ follow redirect และจำกัดขนาด response ที่ 4 MiB
- container รันแบบ non-root บน root filesystem ที่เป็น read-only, ใช้ `no-new-privileges` และเปิด app เฉพาะใน network ภายใน
- credential ของ SQL แยกจาก connection string และ escape ค่าให้อย่างถูกต้อง

14.3 เอกสารเดิมที่ไม่ตรงกับโค้ด

เอกสาร	ข้อความเดิม	ความจริงตามโค้ด
<code>README.md</code> บรรทัดต้น	SQL connector ยังไม่ได้ implement	มี connector แล้ว แต่รอ query/mapping และ read-only verification
<code>README.md</code> หัวข้อ webhook	SQL sweep worker จะ implement ใน Phase 04	ใช้ scheduled approved-SO worker; Hook ไม่ใช่ช่องทางหลัก
<code>README.md</code> สัญญา API	ทุก endpoint ยกเว้น <code>/health</code> ต้องใช้ key	<code>/docs</code> , <code>/openapi.json</code> และ <code>/favicon.ico</code> ก็เป็นสาธารณะ
<code>Integration_Test_Plan.md</code> ERP-04	event_id เดิมที่ข้อมูลเปลี่ยนจะได้ 409	โค้ดตรวจ payload hash และตอบ conflict
<code>Integration_Test_Plan.md</code> VRP-03	มี controlled retry และ attempt record	ทดสอบว่าตั้ง <code>needs_review</code> และไม่ resend อัตโนมัติ; ยังไม่มี attempt history
<code>Thaisausage_Operations_Guide.md</code> หัวข้อ 3	<code>config/local.json</code> ต้องกำหนด source และ approved query set	โค้ดยังไม่ได้อ่าน key เหล่านี้ (F8)

15. เอกสารที่เกี่ยวข้อง

- `docs/ERP_Integration_Guide.md` — สำหรับทีม ERP: webhook, JSON contract และ retry
- `docs/Thaisausage_Operations_Guide.md` — release gates และกติกากการกู้คืน
- `docs/Integration_Test_Plan.md` — test case และแบบฟอร์ม sign-off
- `docs/erp-source-contract.md` — รายการข้อมูลที่ต้องได้จาก ERP/DBA ใน Phase 01
- `docs/do-insert-contract.md` — mapping matrix ของ `tbl_D0hdr` / `tbl_Dodtbl` สำหรับ DBA

- `process/features/erp-sqlserver/active/PHASE_07_D0_INSERT_PLAN_16-09-26.md` — แผน DO Insert
- `docs/openapi.json` และ `https://thaisausage.krs.co.th/docs`
- `process/features/erp-sqlserver/active/ERP_SQLSERVER_PLAN_12-09-26.md` — แผน 6 phase